清華大學
Tsinghua University

2026秋季

计算机系统概论

Introduction to Computer Systems

80X86汇编语言与C语言-2







Zhang，Youhui（张悠慧）

zyh02@tsinghua.edu.cn

计算机体系结构

main.c

```
int array[4] = {1,2,3,4};
static int brray[4] = {1,2,3,4};

static int intra_sum (int x[4],int y)
{
   return x[y-1];
}
int main()
{
    int val = intra_sum(array, 3) + inter_sum(brray,3);
    return val;
}
```

编译

```
intra_sum:
        movslq  %esi, %rsi
        movl    -4(%rdi,%rsi,4), %eax
        ret
main:
        pushq   %rbx
        movl    $3, %esi
        movl    $array, %edi
        call    intra_sum
        movl    %eax, %ebx// val -> ebx
        movl    $3, %esi
        movl    $brray, %edi
        movl    $0, %eax
        call    inter_sum
        addl    %ebx, %eax// val -> eax
        popq    %rbx
        ret
brray:
        .long   1
        .long   2
        .long   3
        .long   4
array:
        .long   1
        .long   2
        .long   3
        .long   4
```

汇编指令

main.o

链接

```
00000000004004de <main>:
  4004de:       53                      push   %rbx
  4004df:       be 03 00 00 00          mov    $0x3,%esi
  4004e4:       bf 40 10 60 00          mov    $0x601040,%edi
  4004e9:       e8 e8 ff ff ff          callq  4004d6 <intra_sum>
  4004ee:       89 c3                   mov    %eax,%ebx
  4004f0:       be 03 00 00 00          mov    $0x3,%esi
  4004f5:       bf 30 10 60 00          mov    $0x601030,%edi
  4004fa:       b8 00 00 00 00          mov    $0x0,%eax
  4004ff:       e8 04 00 00 00          callq  400508 <inter_sum>
  400504:       01 d8                   add    %ebx,%eax
  400506:       5b                      pop    %rbx
  400507:       c3                      retq
```

仅给出部分内容

内存地址

运行

CPU

P C Registers Condition Codes

地址/Addresses

数据/Data

指令/Instructions

Memory

数据段

代码段

C程序在硬件层面的表示

- 数据
  - ……
- 代码
  - 基本概念/基本指令/寻址方式 / 程序控制流与相关指令（第三讲）
  - ……

程序/数据在硬件层面的表示与运行



目录

CONTENTS

01 控制流、条件码与相关指令

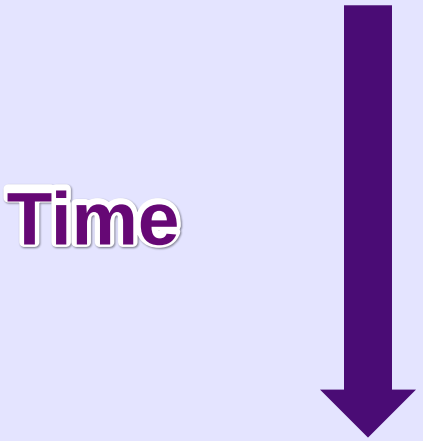
02 while / do-while / for 循环

03 switch语句

控制流 (control flow)

处理器只做一件事:

- 从启动到关闭，CPU读取和执行(解释)一个指令序列，一次一条
- 这个执行顺序被称为CPU的控制流



本部分主要介绍改变控制流的两种机制:

- 无条件跳转（jump） / 条件跳转（branch）
- 过程调用与返回（call / return）

处理器状态 (x86-64, 部分)

当前执行程序的信息

- **临时数据**
  (%rax, …)
- **栈顶地址**
  (%rsp)
- **当前指令地址（下一条）**
  (%rip, …)
- **条件码**
  (CF, ZF, SF, OF)



条件码（由指令隐式设置）

回想下“2.1讲”提出的问题——**C语言层面的无符号/带符号的语义区分由谁来完成**

- **CF　进/借位标志**
- **ZF　零标志**
- **SF　符号标志**
- **OF　溢出标志**

这些条件码由算术指令隐含设置

- **addq Src, Dest**
  - **CF 有进位则置1**　可用于检测无符号整数运算的溢出
  - **SF 计算结果小于0则置1**
  - **OF 补码运算溢出（即带符号整数运算）则置1**　(a>0 && b>0 && t<0) || (a<0 && b<0 && t>=0)
  - **ZF 计算结果为0则置1**
- **addl Src, Dest**

leaq 指令不设置条件码

比较（Compare）指令

cmpq Src2,Src1　　**cmpl Src2,Src1**

cmpq b,a 类似于计算a-b（但是不改变目的操作数）

CF 若最高位有借位则置1

- 可用于无符号数的比较

ZF 若a == b则置1　　**SF 若 (a-b) < 0则置1**

OF 若补码计算溢出则置1

- (a>=0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)

测试（Test）指令

testq Src2,Src1　　testl Src2,Src1

- 计算Src1 & Src2并设置相应的条件码，但是不改变目的操作数

- ZF：若Src1 & Src2 == 0则置1
- SF：若Src1 & Src2 < 0则置1

- test指令使CF、OF为0

读取条件码

SetX 指令：

- 读取当前的条件码（或者某些条件码的组合），并存入目的字节寄存器

SetX	涉及的条件码（组合）	语义
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~(SF^OF)&~ZF	Greater (Signed)
setge	~(SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF)|ZF	Less or Equal (Signed)
seta	**~CF&~ZF**	**Above (unsigned)**
setb	**CF**	**Below (unsigned)**

如果补码减法计算溢出，则OF置1(a>=0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)

回顾：x86-64 通用寄存器 （低8位单独命名，可独立访问）

%rax	%al
%rbx	%bl
%rcx	%cl
%rdx	%dl
%rsi	%sil
%rdi	%dil
%rsp	%spl
%rbp	%bpl

%r8	%r8b
%r9	%r9b
%r10	%r10b
%r11	%r11b
%r12	%r12b
%r13	%r13b
%r14	%r14b
%r15	%r15b

SetX 指令

读取当前的条件码（或者某些条件码的组合），并存入目的“字节”寄存器

- **余下的7个字节不会被修改**
- **通常使用“movzbl”指令对目的寄存器进行“0”扩展**

```
int gt (long x, long y)
{
   return x > y;
}
```

寄存器	用途
%rdi	Argument x
%rsi	Argument y
%rax	Return value

```
cmpq    %rsi, %rdi    # Compare x:y
setg    %al           # Set when >
movzbl  %al, %eax     # Zero rest of %rax
ret
```

X86-32架构下，16位操作数指令产生了16位结果，那么对应的目标寄存器的高16位如何处理?

“64-bit operands generate a 64-bit result in the destination general-purpose register. 32-bit operands generate a 32-bit result, zero-extended to a 64-bit result in the destination general-purpose register.”

摘自”Intel® 64 and IA-32 Architectures Software Developer’s Manual Volume 1: Basic Architecture”

跳转指令

jX 指令　依赖当前的条件码选择下一条执行语句（是否顺序执行）

jX	涉及的条件码（组合）	语义
jmp	1	Unconditional
je	ZF	Equal / Zero
jne	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF)&~ZF	Greater (Signed)
jge	~(SF^OF)	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF)|ZF	Less or Equal (Signed)
ja	~CF&~ZF	Above (unsigned)
jb	CF	Below (unsigned)

条件跳转示例 (Old Style)



编译命令

gcc –Og -S -fno-if-conversion control.c

```
long absdiff
  (long x, long y)
{
   long result;
   if (x > y)
     result = x-y;
   else
     result = y-x;
   return result;
}
```

```
absdiff:
   cmpq    %rsi, %rdi   # x-y
   jle     .L4
   movq    %rdi, %rax
   subq    %rsi, %rax
   ret
.L4:                    # x <= y
   movq    %rsi, %rax
   subq    %rdi, %rax
   ret
```

寄存器	用途
%rdi	参数x
%rsi	参数y
%rax	返回值

Goto风格的C代码表示

```
long absdiff
  (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
long absdiff_j
  (long x, long y)
{
    long result;
    int ntest = (x <= y);
    if (ntest) goto Else;
    result = x-y;
    goto Done;
 Else:
    result = y-x;
 Done:
    return result;
}
```

条件表达式

C 代码

```
val = Test ? Then_Expr : Else_Expr;
```

```
val = x>y ? x-y : y-x;
```

Goto风格的C代码

```
        ntest = !Test;
        if (ntest) goto Else;
        val = Then_Expr;
        goto Done;
Else:
        val = Else_Expr;
Done:
        . . .
```

- 为*then*与 *else*表达式分别创建独立的代码区域
- 根据*test*结果执行适当的一个

条件移动指令

条件移动

- 语义: if (Test) Dest ← Src
- 1995年以后的x86处理器都支持此类指令
- GCC编译器在确保“安全”的前提下会使用它们

Why?

- 条件跳转指令对于现代流水线处理器的执行效率有很大的负面影响
- 条件移动指令可以避免这一现象

C代码

```
val = Test
    ? Then_Expr
    : Else_Expr ;
```

Goto风格的代码表示

```
result = Then_Expr ;
eval = Else_Expr ;
nt = !Test;
if (nt) result = eval;
return result;
```

条件移动指令

```
long absdiff (long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

寄存器	用途
%rdi	参数x
%rsi	参数y
%rax	返回值

absdiff:

```
movq    %rdi, %rax      # x
subq    %rsi, %rax      # result = x-y
movq    %rsi, %rdx
subq    %rdi, %rdx      # eval = y-x
cmpq    %rsi, %rdi      # x:y
cmovle  %rdx, %rax      # if <=, result = eval
ret
```

gcc –O2 / –O1

微体系结构背景*

处理器流水线（五级流水示例）

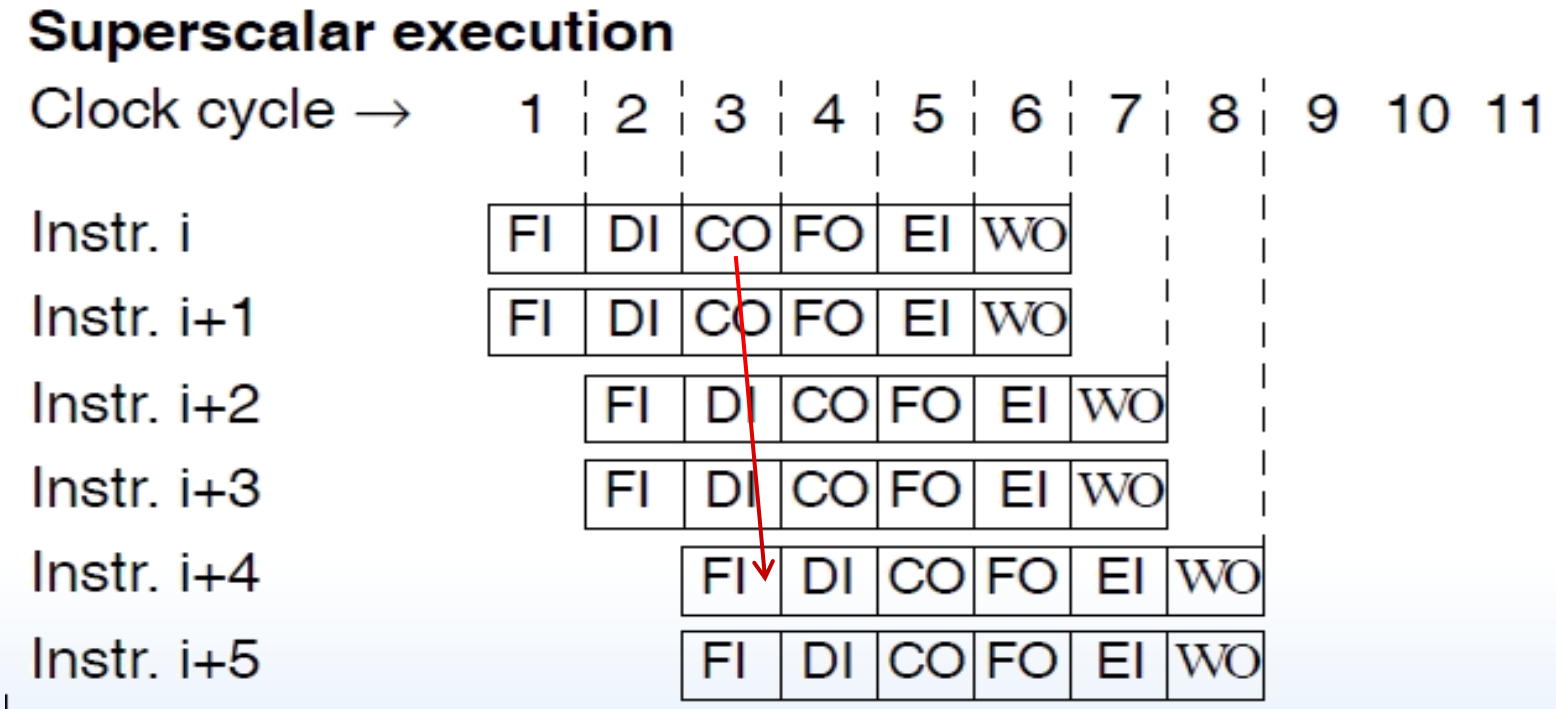
- 指令获取段 **(IF)**
- 算术操作段 **(ALU)**
- 写回段 **(WB)**
- 寄存器读取段 **(RD)**
- 内存访问段 **(MEM)**

运行周期



微体系结构背景*

现代的通用处理器 支持深度流水线以及多发射结构，
如奔腾4处理器有20多级流水线，同时最多可以有126条指令在运行



条件跳转指令往往会引起一定的性能损失，因此需要尽量消除

条件转移指令的局限性

```
val  = Then-Expr;
vale = Else-Expr;
val  = vale if !Test;
```

```
int xgty  = 0, xltey = 0;
int absdiff_se(
     int x, int y)
{
     int result;
     if (x > y) {
          xgty++;  result = x-y;
     } else {
          xltey++; result = y-x;
     }
     return result;
}
```

限制使用的场合

1. Then-Expr 或Else-Expr 表达式有“副作用”

2. Then-Expr 或 Else-Expr 表达式的计算量较大



填空题 3分

设置

```
int test(long x, long y)
{
    return (x > y) ? x+y : x-y;
}
```

```
test:
        movl    %esi, %eax
        [填空1]  %eax
        cmpq    %rsi, %rdi
        cmovg   [填空2],[填空3]
        addl    %edi, %eax
        ret
```

作答

例题

```
#include <stdlib.h>

#define likely(x)      __builtin_expect(!!(x), 1)
#define unlikely(x)  __builtin_expect(!!(x), 0)
int main(int argc, char *argv[])
{
        int a,b;
       /* Get the value from somewhere GCC can't optimize */
        a = atoi (argv[1]); /*string to int*/
        b = a*a;
        if (unlikely (a == 2))
        // if (likely (a == 2))
        {
          a++;  b++;
        }
        else
        {
          a--;  b--;
        }
        return a+b;
}
```

main:

```
subq    $8, %rsp
movq    8(%rsi), %rdi
xorl    %esi, %esi
movl    $10, %edx
call    strtol              #atoi调用，
                            #返回值在eax中
movl    %eax, %esi  #esi = a
movl    $3, %ecx    #ecx = 3
imull   %eax, %esi  #esi (b) = a²
cmpl    $2, %eax
leal    1(%rsi), %edx   #edx = b++
je      .L3
leal    -1(%rax), %ecx  #ecx = a--
leal    -1(%rsi), %edx  #edx = b--
```

.L3:

```
leal    (%rcx,%rdx), %eax
addq    $8, %rsp
ret
```

gcc –O2

例题

```
main:
        subq    $8, %rsp
        movq    8(%rsi), %rdi
        xorl    %esi, %esi
        movl    $10, %edx
        call    strtol
        movl    %eax, %ecx
        imull   %eax, %ecx
        cmpl    $2, %eax
        jne     .L2
        leal    1(%rcx), %eax
        movl    $3, %edx
.L3:
        addl    %edx, %eax
        addq    $8, %rsp
        ret
.L2:
        leal    -1(%rax), %edx
        leal    -1(%rcx), %eax
        jmp     .L3
```

问题

分析编译指示（directives）

> #define likely(x) __builtin_expect(!!(x), 1)
> #define unlikely(x) __builtin_expect(!!(x), 0)

的作用——为何生成的指令序列的顺序不同，与处理器流水线的运行过程有何关系?



“Do-While” 循环示例

C代码

```
long pcount_do
  (unsigned long x) {
  long result = 0;
  do {
      result += x & 0x1;
      x >>= 1;
  } while (x);
   return result;
}
```

Goto风格的C代码

```
long pcount_goto
   (unsigned long x) {
      long result = 0;
  loop:
      result += x & 0x1;
      x >>= 1;
      if(x) goto loop;
      return result;
}
```

- 计算参数x的二进制表示中的1的个数
- 使用条件分支来继续循环或退出循环

“Do-While” 循环的编译后代码

Goto风格的C代码

```
long pcount_goto
    (unsigned long x) {
    long result = 0;
 loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

寄存器	用途
%rdi	参数x
%rax	result

```
    movl    $0, %eax            #  result = 0
.L2:                            # loop:
    movq    %rdi, %rdx
    andl    $1, %edx            #  t = x & 0x1
    addq    %rdx, %rax          #  result += t
    shrq    %rdi                #  x >>= 1
    jne     .L2                 #  if (x) goto loop
    ret
```

通用的“Do-While” 转换

C代码

```
do
    Body
while (Test);
```

Goto风格的C代码

```
    loop:
        Body
        if (Test)
            goto loop
```



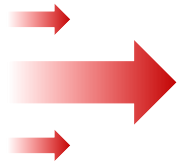
Body

```
{
    Statement 1;
    Statement 2;
      …
    Statement n;
}
```

通用的“While” 转换-1

While风格

```
while (Test)
    Body
```



Goto风格

```
    goto test;
loop:
    Body
test:
    if (Test)
        goto loop;
done:
```

- “Jump-to-middle” 转换方式
- 编译时使用的命令行参数：`-Og`

While 循环示例

C 代码

```
long pcount_while
  (unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Goto风格的C代码

```
long pcount_goto_jtm
  (unsigned long x) {
    long result = 0;
    goto test;
 loop:
    result += x & 0x1;
    x >>= 1;
 test:
    if(x) goto loop;
    return result;
}
```

- 请与do-while代码进行比较
- 通过初始的*goto*语句跳转到标号“test”，开始循环

通用的“While” 转换-2

While风格

```
while (Test)
  Body
```

Do-While风格

```
if (!Test)
    goto done;
  do
    Body
  while(Test);
done:
```

- 等价的**do-while**代码（左下框图所示）
- 编译时使用的命令行参数：`-O1`

Goto风格

```
if (!Test)
        goto done;
loop:
    Body
    if (Test)
        goto loop;
done:
```

While循环示例

C代码

```
long pcount_while
    (unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Goto风格

```
long pcount_goto_dw
    (unsigned long x) {
      long result = 0;
      if (!x) goto done;
 loop:
      result += x & 0x1;
      x >>= 1;
      if(x) goto loop;
 done:
      return result;
}
```

- 请与do-while代码进行比较
- 由初始条件判断是否进入循环

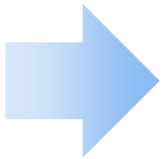
“For” 循环的形式

一般的 for 循环形式

for (*Init*; *Test*; *Update*)

Body

```
#define WSIZE 8*sizeof(int)
long pcount_for
        (unsigned long x)
{
        size_t i;
        long result = 0;
        for (i = 0; i < WSIZE; i++)
        {
                unsigned bit =
                        (x >> i) & 0x1;
                result += bit;
        }
        return result;
}
```



Init	i = 0
Test	i < WSIZE
Update	i++
Body	{ unsigned bit = (x >> i) & 0x1; result += bit; }

“For” 循环→ While 循环

For风格

```
for (Init; Test; Update )
    Body
```

While风格

```
Init;
while (Test ) {
        Body
        Update;
}
```



填空题 3分

设置

```
test:
        testl   %edi, %edi
        js      .L6
        movl    $0, %edx
        movl    $0, %eax
.L5:
        leal    2(%rax), %ecx
        leal    (%rax,%rax), %r8d
        cmpl    %eax, %esi
        movl    %ecx, %eax
        cmovle  %r8d, %eax
        addl    $2, %edx
        cmpl    %edx, %edi
        jge     .L5
        ret
.L6:
        movl    $0, %eax
        ret
```

```
int test(int val, int input)
{
    int local = 0;
    int i = 0;
    for(i=0; i [填空1] val; i+= [填空2] )
    {
        if (local < input)
            local+=2;
        else
            [填空3] ;
    }
    return local;
}
```

作答

Switch语句

依据不同情况来采用不同的实现技术

- 使用一组if-then-else语句来实现
- 使用跳转表（jump table）
  - （建议）编译开关：--param case-values-threshold=1



Switch语句示例

```
long switch_eg
   (long x, long y, long z)
{
    long w = 1;
    switch(x) {
    case 1:
            w = y*z;
            break;
    case 2:
            w = y/z;
            /* Fall Through */
    case 3:
            w += z;
            break;
    case 5:
    case 6:
            w -= z;
            break;
    default:
            w = 2;
    }
    return w;
}
```

Multiple case labels

case 5 & 6

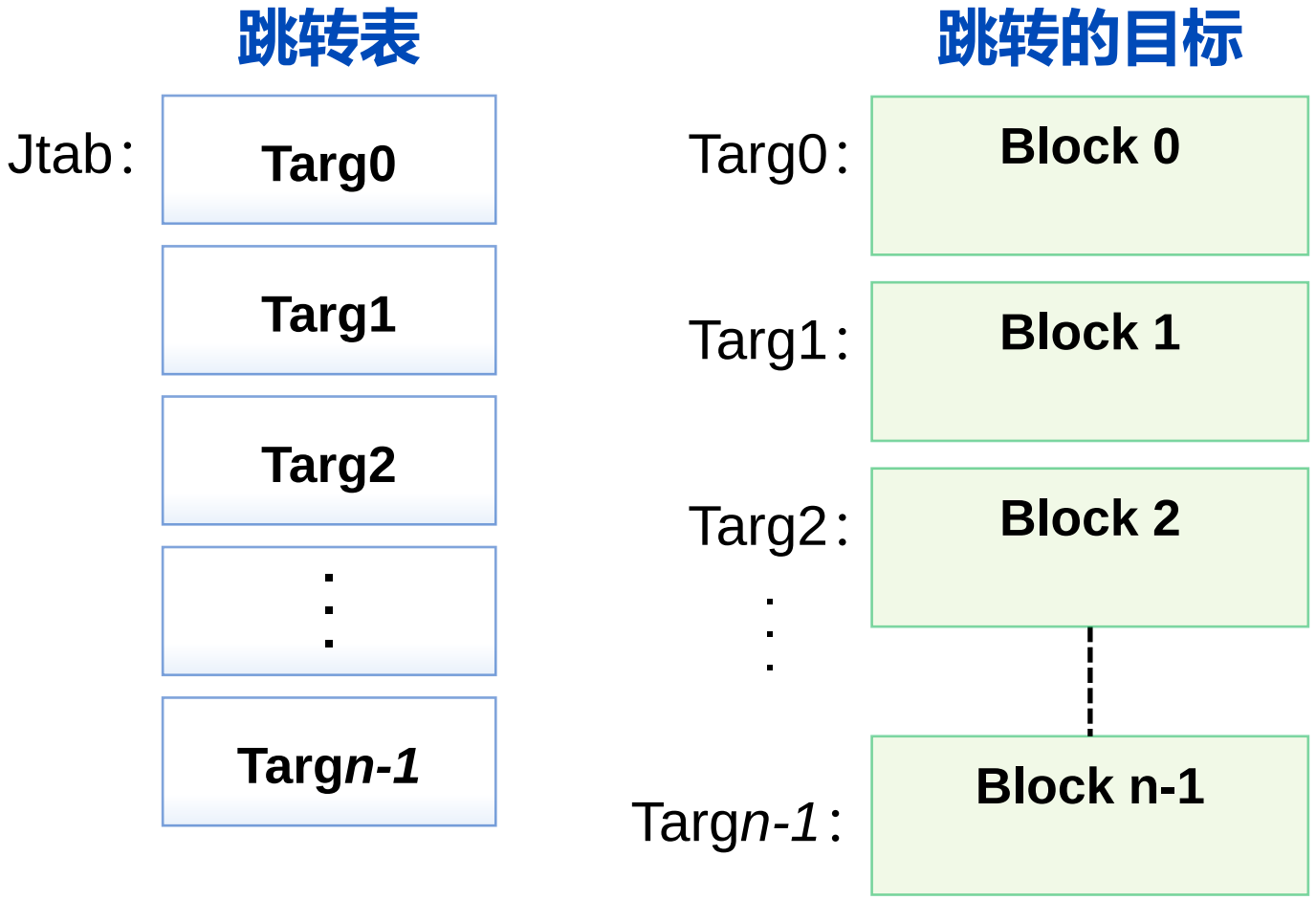
Fall through cases

case 2

Missing cases

case 4

switch case代码 - 跳转表



Switch风格的代码

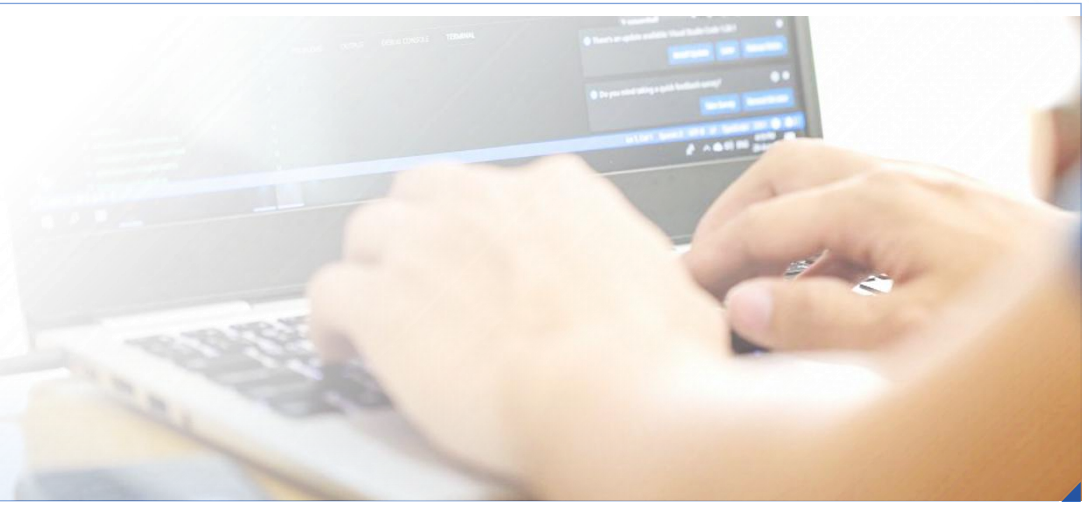
```
switch(x) {
  case val_0:
        Block 0
  case val_1:
        Block 1
        • • •
  case val_n-1:
        Block n–1
        }
```

用goto风格的代码表示

```
goto *JTab[x];
```

Switch语句示例

```
long switch_eg(long x, long y, long z)
{
        long w = 1;
        switch(x) {
          . . .
        }
        return w;
}
```



初始化:

```
switch_eg:
        movq    %rdx, %rcx
        cmpq    $6, %rdi    # x:6
        ja      .L8
        jmp     *.L4( ,%rdi,8)
```

why? ⟶ ja .L8

寄存器	用途
%rdi	参数x
%rsi	参数y
%rdx	参数z
%rax	返回值
注意: w没有在这儿初始化	

Switch语句示例

```
long switch_eg(long x, long y, long z)
{
        long w = 1;
        switch(x) {
          . . .
        }
        return w;
}
```

初始化:

```
switch_eg:
    movq    %rdx, %rcx
    cmpq    $6, %rdi        # x:6
    ja      .L8             #
    jmp     *.L4(,%rdi,8)   # goto *JTab[x]
```

Indirect jump (间接跳转)

跳转表（jump table）

```
.section    .rodata
.align 8
.L4:
    .quad    .L8    # x = 0
    .quad    .L3    # x = 1
    .quad    .L5    # x = 2
    .quad    .L9    # x = 3
    .quad    .L8    # x = 4
    .quad    .L7    # x = 5
    .quad    .L7    # x = 6
```

跳转表的构建与访问

跳转表的结构

- 每一表项（即目标地址）占据8字节
- 表的基地址为 .L4

跳转指令解析

- **直接跳转:** jmp .L8
- 跳转到标号.L8表示的地址
- **间接跳转:** jmp *.L4(,%rdi,8)
- 表的基地址为.L4
- 每一表项占据8字节
- **存储“跳转目标地址”的地址**的计算方式是 .L4 + %rdi*8
  - 0 ≤ %rdi ≤ 6

跳转表

```
.section    .rodata
.align 8
.L4:
    .quad    .L8    # x = 0
    .quad    .L3    # x = 1
    .quad    .L5    # x = 2
    .quad    .L9    # x = 3
    .quad    .L8    # x = 4
    .quad    .L7    # x = 5
    .quad    .L7    # x = 6
```

跳转表详析

跳转表

```
.section    .rodata
.align 8
.L4:
    .quad    .L8    # x = 0
    .quad    .L3    # x = 1
    .quad    .L5    # x = 2
    .quad    .L9    # x = 3
    .quad    .L8    # x = 4
    .quad    .L7    # x = 5
    .quad    .L7    # x = 6
```

```
switch(x) {
case 1:                 // .L3
        w = y*z;
        break;
case 2:                 // .L5
        w = y/z;
        /* Fall Through */
case 3:                 // .L9
                        w += z;
        break;
case 5:
case 6:                 // .L7
        w -= z;
        break;
default:                // .L8
        w = 2;
}
```

x为1时对应的代码

```
        switch(x) {
        case 1:                    // .L3
            w = y*z;
             break;
         . . .
}
```

```
.L3:
        movq        %rsi, %rax   # y
        imulq       %rdx, %rax   # y*z
        ret
```

寄存器	用途
%rdi	参数x
%rsi	参数y
%rdx	参数z
%rax	返回值



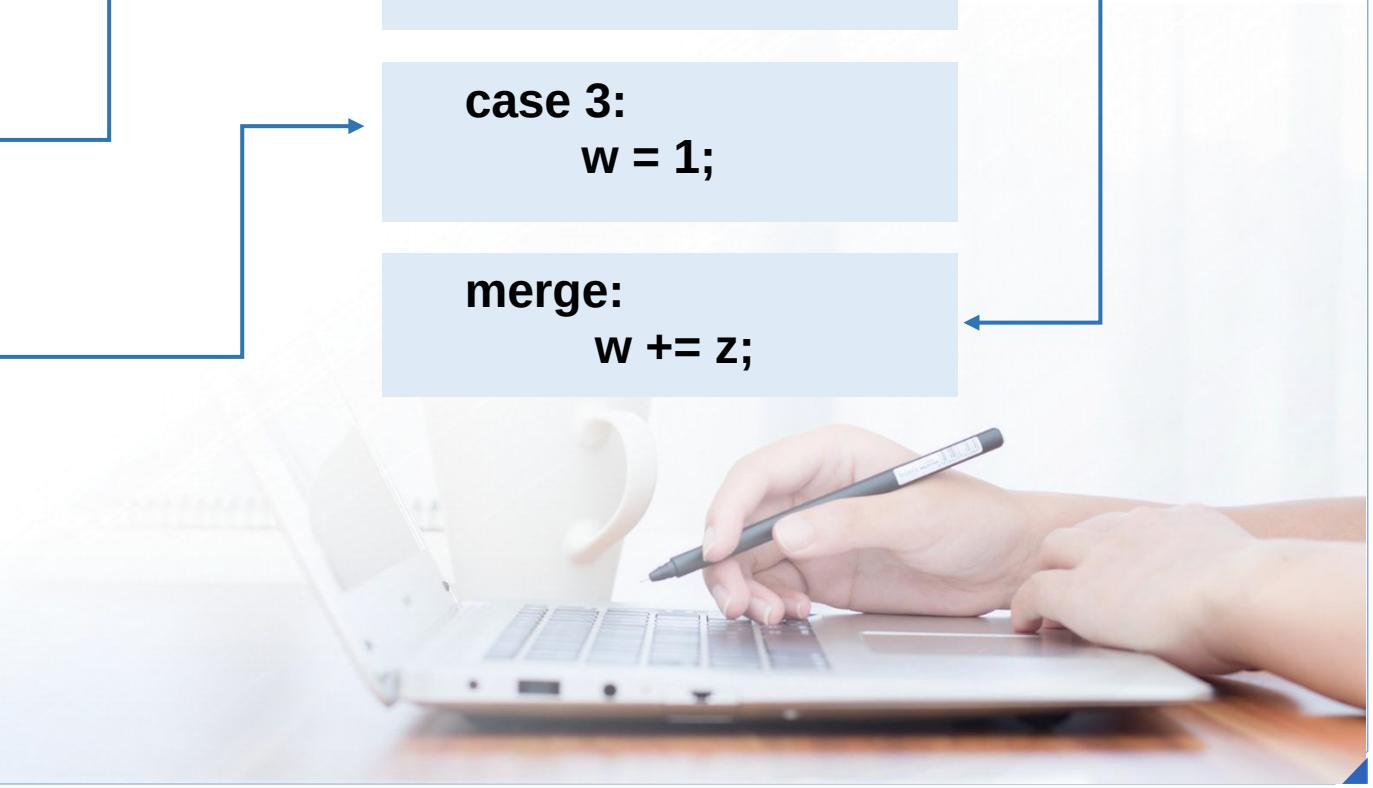
处理 Fall-Through

```
long w = 1;
        . . .
switch(x) {
        . . .
case 2:
        w = y/z;
        /* Fall Through */
case 3:
        w += z;
        break;
        . . .
}
```

```
case 2:
        w = y/z;
        goto merge;

case 3:
        w = 1;

merge:
        w += z;
```



x为2或3时对应的代码

```
long w = 1;
        . . .
switch(x) {
        . . .
case 2:
    w = y/z;
    /* Fall Through */
case 3:
    w += z;
    break;
        . . .
}
```

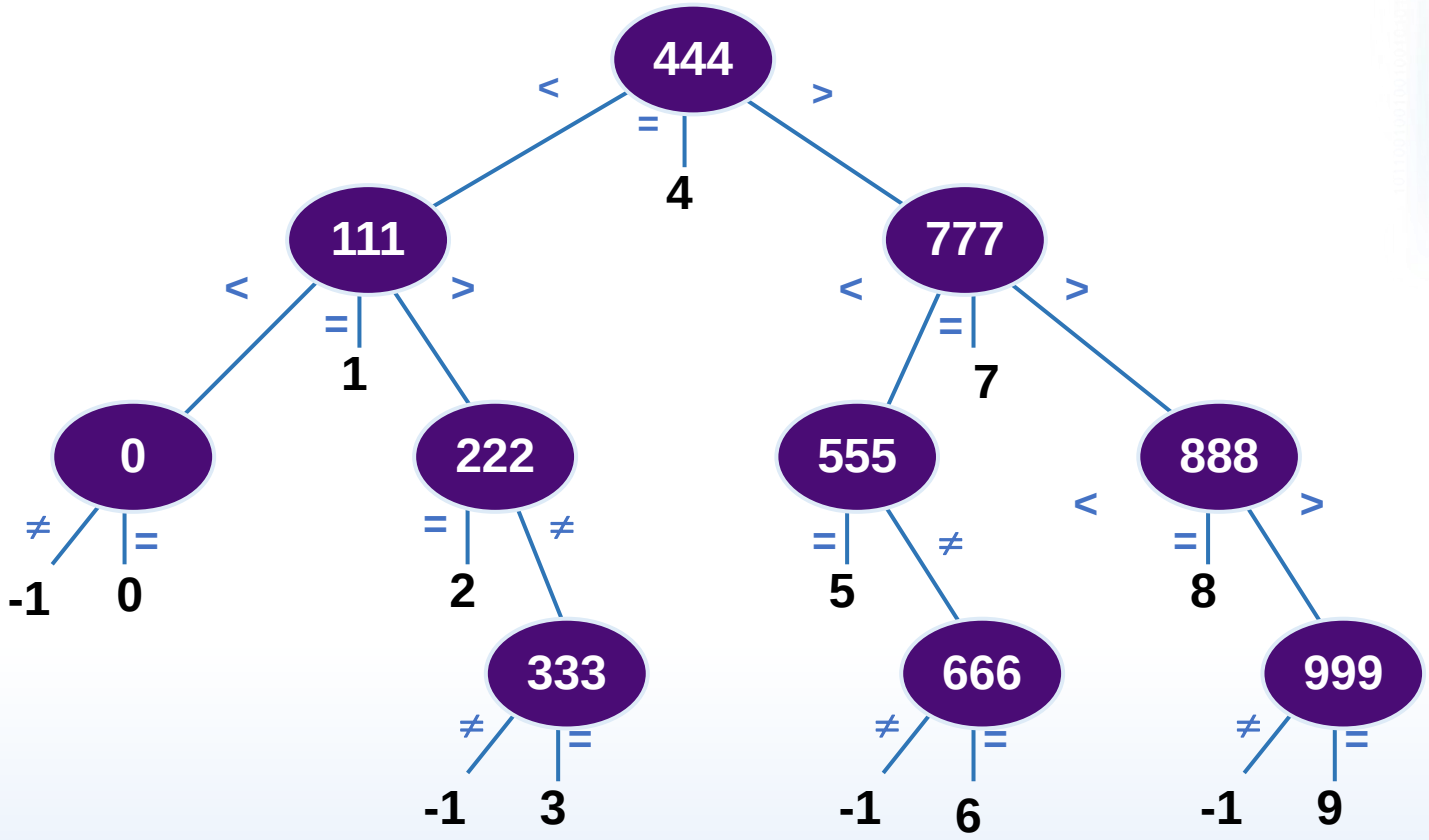
```
.L5:                                    # Case 2
        movq        %rsi, %rax
        cqto                            # sign extend rax to
                                        # rdx:rax
        idivq       %rcx                #  y/z
        jmp         .L6                 #  goto merge
.L9:                                    # Case 3
        movl         $1,%eax            #  w = 1
.L6:                                    # merge:
        addq        %rcx, %rax          #  w += z
        ret
```

Signed divide RDX:RAX by *r/m64*, with result stored in RAX ← Quotient, RDX ← Remainder.

寄存器	用途
%rdi	参数x
%rsi	参数y
%rdx	参数z
%rax	返回值

跳转表的适用范围

其它情况下会使用平衡二叉树的结构组织，提升性能



小 结

01 控制流、条件码与相关指令

- 设置 / 读取 / 条件跳转指令 / 条件传送指令
- if-then-else

02 while / do-while / for 循环

03 switch语句

关于“定位”

跳转指令的目的如何定位?

- 直接跳转（形如jmp .L8）
  - 间接跳转解决方案不一样（形如jmp *.L4(,%rdi,8)），需要Linker解决

```
0000000000000023 <main>:
 …          …                    …..
 53:        89 45 ec             mov   %eax,-0x14(%rbp) # val
 56:        83 7d ec 00          cmpl  $0x0,-0x14(%rbp)  # val>=0 ?
 5a:        78 05                js    61                # no
 5c:        8b 45 ec             mov   -0x14(%rbp),%eax # yes; %eax = val
 5f:        eb 05                jmp   66
 61:        b8 ff ff ff ff       mov   $0xffffffff,%eax  # %eax = -1
 66:        48 83 c4 18          add   $0x18,%rsp
 6a:        5b                   pop   %rbx
 6b:        5d                   pop   %rbp
 6c:        c3                   retq
```

```
int main()
{
   int val = …;

   if (val>=0)
            return val;
   else
            return -1;
}
```

机器指令: eb表示无条件跳转jmp，78表示js，后面跟的数字与目标跳转地址相关

问题回顾（补充内容）

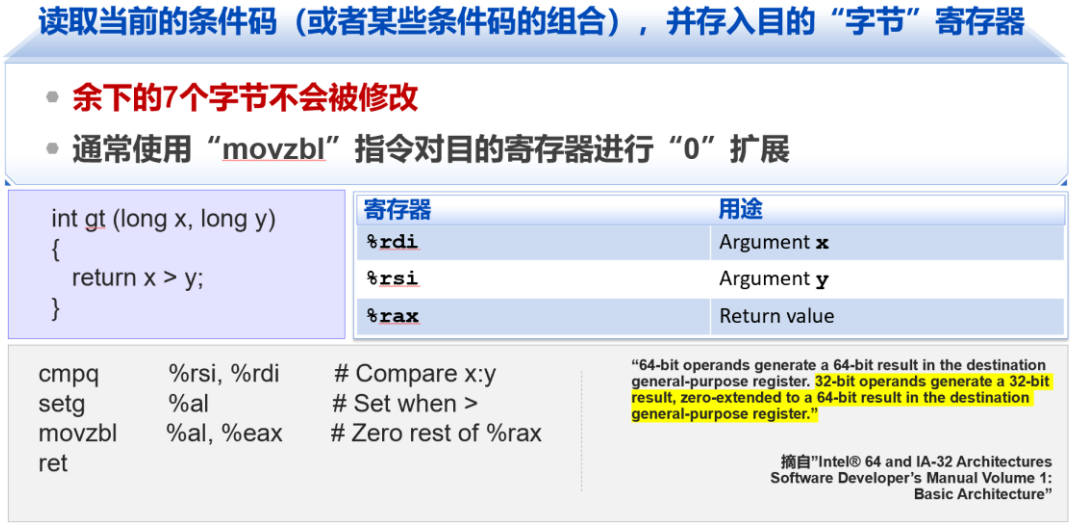
x86-64架构下，32位计算指令对于目标寄存器高32位的副作用

32位下16位计算指令是否有类似表现？Why?

某大模型*错误回答：80386不零扩展16位操作，是因为 8086的AX→AH/AL拆分有独立名称且被大量代码依赖，改变行为会破坏兼容性；x86-64零扩展32位操作，是因为RAX的高32位没有独立名称、64位模式是全新原生架构、且借机消除了部分寄存器更新的硬件复杂性

- 经过几轮对话后，校正

*LLM不断进化，这儿作为一个AI输出结果需要批判看待的例子



Partial Data Dependence 源于“寄存器的部分写入”与“寄存器的完整读取”之间的语义差距——8/16 位写入保留高位，导致后续读取需合并新旧值，会引入 merging μop 和流水线停滞；x86-64 的 movl 零扩展设计用"完整 32 位更新（高 32 位清零）“ 消除了这一依赖